LAPORAN PRAKTIKUM

INTERNET OF THINGS II

TUGAS 04



Nama :

Nama : Fadillah Akbar

NPM : 25782039

Kelas : TRI 3B

Dosen Pengampu : **Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.**

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

JURUSAN TEKNOLOGI INFORMASI

TEKNOLOGI REKAYASA INTERNET

POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

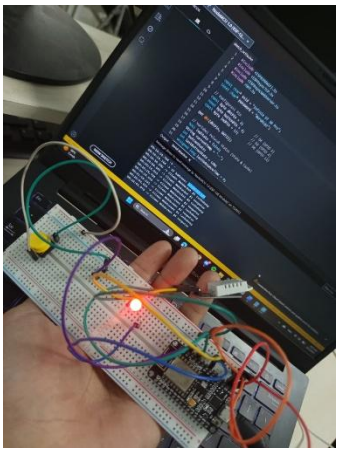
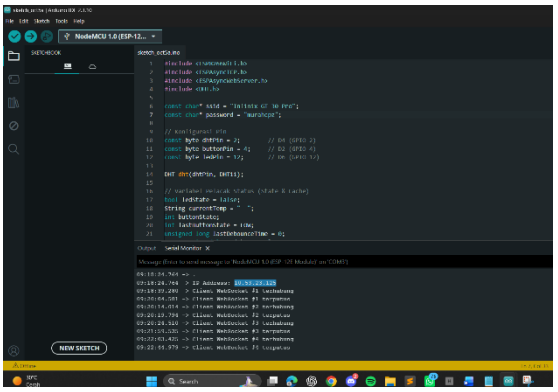
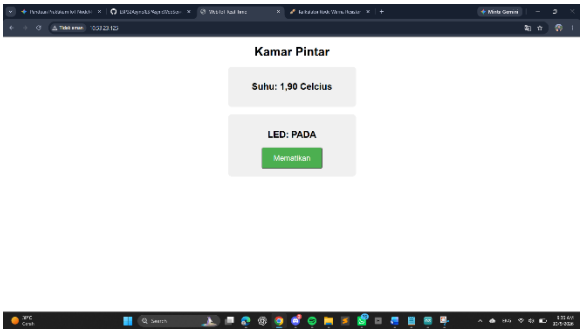


Foto Alat



Ss an Se rial Monitor



Hasil WEB

4.6 Latihan

1. **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab:

- Efisiensi Bandwidth/Data: Pada metode Meta Refresh HTTP, browser harus berulang kali mengirimkan HTTP Request Header yang berukuran cukup besar dan mengunduh ulang seluruh struktur dokumen HTML, CSS, serta Script dari awal. Sementara pada WebSocket, setelah proses handshake awal selesai, saluran komunikasi dua arah (full-duplex) tetap terbuka secara persisten. Data dikirim hanya dalam bentuk payload mentah berukuran sangat kecil (beberapa byte teks JSON) tanpa overhead header HTTP yang besar.
- Tampilan Tidak Berkedip: Meta Refresh memaksa browser melakukan reload/muat ulang seluruh halaman web dari nol, sehingga DOM dirender ulang dan memicu efek flicker (berkedip). Sebaliknya, WebSocket bekerja secara asynchronous di latar belakang. Data JSON yang diterima ditangkap oleh JavaScript untuk memperbarui teks pada elemen DOM spesifik secara langsung (innerHTML) tanpa perlu menyegarkan atau memuat ulang halaman.

2. **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Jawab:

Penggunaan delay(3000) di dalam loop() akan menghentikan (*blocking*) eksekusi prosesor NodeMCU selama 3 detik penuh. Bahaya sistematis yang terjadi meliputi:

- **Kehilangan Real-Time Respon:** Tekanan pada push button mekanik yang terjadi saat delay() berlangsung tidak akan terbaca oleh sistem.
- **Keterlambatan/Latensi Komunikasi WebSocket:** Sinyal toggle dari browser tidak dapat diolah seketika oleh NodeMCU, menyebabkan status tombol web dan indikator tidak sinkron (*lag*).

- **Potensi Disconnect & Watchdog Reset (WDT):** Komunikasi background TCP/IP dan antrean paket WebSocket terhambat. Jika *stack* jaringan tidak diproses secara teratur, modul ESP8266 dapat mengalami *timeout* koneksi atau ter-reset otomatis oleh hardware Watchdog Timer (Soft WDT reset).

3. **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawab:

Target ID elemen HTML yang diubah oleh fungsi JavaScript pada blok tersebut adalah:

- **tempValue** → Menyuntikkan teks angka suhu terbaru dari paket data JSON (dataObj.suhu).
- **ledStatus** → Mengubah teks indikator status LED menjadi **"ON"** atau **"OFF"**.
- **toggleBtn** → Mengubah teks label tombol menjadi **"Turn OFF"** atau **"Turn ON"**, serta mengganti warna/gaya tombol melalui btn.className (btn-on atau btn-off).

4. **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
    a. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).

Jawab:

- Tambah Variabel Global Penampung Kelembapan:

```
String currentHum = "--"; // Penampung cache kelembapan
```

- Modifikasi Fungsi notifyClients() (JSON Payload):

```
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\"}"; // Menyisipkan data kelembapan
  ws.textAll(jsonString);
}
```

- Modifikasi Pembacaan DHT pada loop() (Timer Non-Blocking millis()):

```
// Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
if ((millis() - lastTime) > 3000) {
  float t = dht.readTemperature();
  float h = dht.readHumidity(); // Baca kelembapan

  if(!isnan(t) && !isnan(h)) {
    currentTemp = String(t);
    currentHum  = String(h); // Simpan ke variabel cache
    notifyClients();         // Push pembaruan suhu & kelembapan otomatis
  }
  lastTime = millis();
}
```

b. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:

- Penambahan Elemen Kartu HTML:

```
<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>
```

- Modifikasi Fungsi JavaScript onMessage(event):

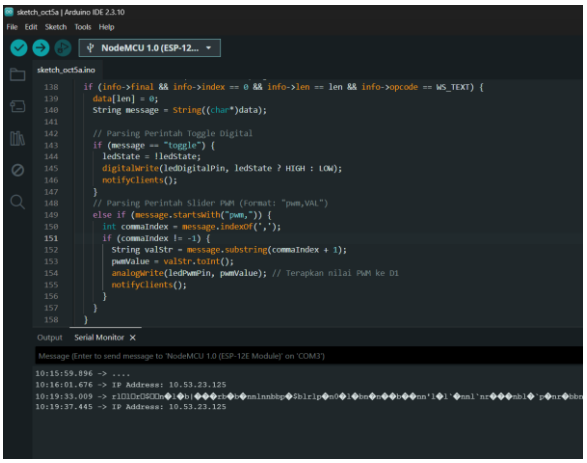
```
function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
  if(dataObj.suhu !== undefined) {
     document.getElementById('tempValue').innerHTML = dataObj.suhu;
  }

  // Menyuntikkan teks angka Kelembapan ke dalam id "humValue" HTML
  if(dataObj.hum !== undefined) {
     document.getElementById('humValue').innerHTML = dataObj.hum;
  }

  // Merombak UI Tombol & Teks sesuai status hardware
  if(dataObj.led !== undefined) {
     var btn = document.getElementById('toggleBtn');
     var status = document.getElementById('ledStatus');
     if(dataObj.led == "1"){
       status.innerHTML = "ON";
       btn.innerHTML = "Turn OFF";
       btn.className = "btn-on";
     } else {
       status.innerHTML = "OFF";
       btn.innerHTML = "Turn ON";
       btn.className = "btn-off";
     }
  }
}
```

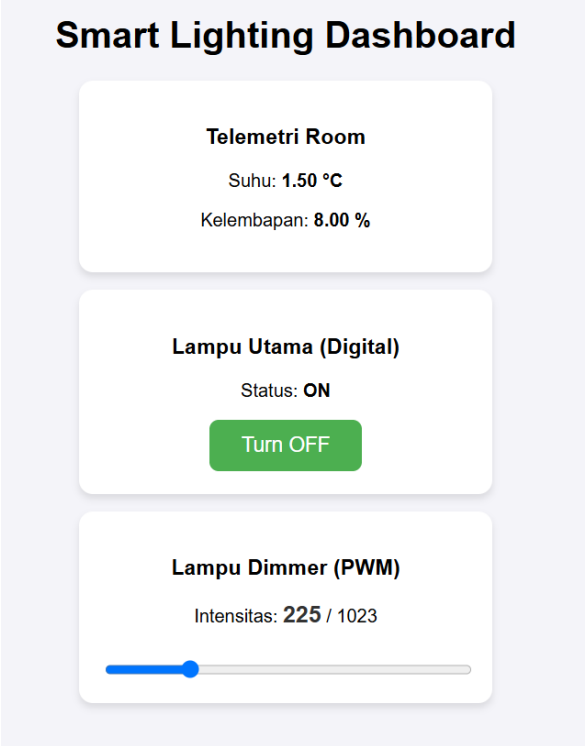
4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*



SS an Serial Monitor



Foto Alat



SS an Tampilan Web